

TRANSLINGUA : AI-POWERED MULTI-LANGUAGE TRANSLATOR

Google Cloud Generative AI

MENTOR NAME : SRI LANKA LAKSHMI NARAYANA SIR

FACULTY MENTOR NAME : K.RATNA KUMARI MADAM

TEAM ID : LTVIP2026TMIDS38652

TEAM LEADER : TUMMALAPUDI MANIKANTA



ID NO : SBAP0032708

MAIL ID: manikantatummalapudi03@gmail.com

TEAM MEMBERS:

1. ADDALA DURGA PRASAD



ID NO ; SBAP0032716

MAIL ID : prasadnaidu293293@gmail.com

2. MOHAMMAD ABDUL RASHAAD



ID NO : SBAP0032740

MAIL ID : abdulrashaad1234@gmail.com

INTRODUCTION

Project Description:

TransLingua is a cutting-edge web application designed to harness the power of advanced AI to provide seamless language translation services. Built using Streamlit and Google's Generative AI, TransLingua offers an intuitive and user-friendly interface for translating text between multiple languages. By simply inputting text and selecting the desired source and target languages, users can instantly receive accurate translations powered by the latest AI models. This tool is ideal for anyone needing reliable and fast translations, whether for personal, educational, or professional purposes.

Scenario 1: Global Business Expansion

A company planning to expand its market reach to non-English speaking regions needs to translate business documents, marketing materials, and customer communications into multiple languages. TransLingua enables the company to quickly translate promotional content and technical documents, ensuring consistency and accuracy across different languages. This helps the company effectively communicate with a broader audience and tailor its messaging to local markets.

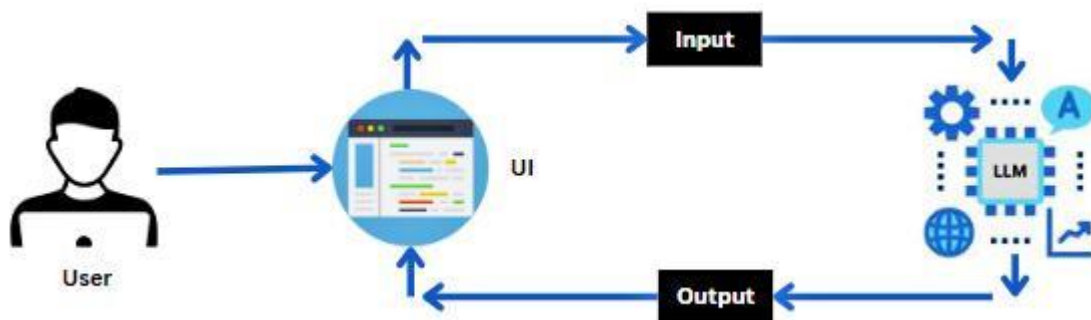
Scenario 2: Academic Research and Collaboration

Researchers and academics collaborating on international projects often need to translate research papers, academic articles, and communication between team members who speak different languages. TransLingua assists in translating complex academic texts and facilitating cross-border collaborations by providing accurate translations of scholarly documents.

Scenario 3: Travel and Tourism Assistance

Travelers visiting foreign countries often face language barriers when trying to communicate with locals or understand travel-related information. TransLingua serves as a handy tool for translating signs, menus, and other text encountered during travel. This enhances the travel experience by making it easier for tourists to navigate unfamiliar environments and interact with local residents.

Architecture



Project Flow

Users input the text they want to translate, select the source language, and choose the target language using the Streamlit UI.

- The input text and language selections are sent to the translation backend, which utilizes an AI-driven translation model to process the request.
- The AI model translates the text from the source language to the target language, providing an accurate and contextually relevant translation.
- The translated text is formatted and refined by the AI to ensure clarity and coherence in the target language.
- The translated text is sent back to the frontend of the Streamlit app for display to the user.
- Users can review the translated text, make additional modifications if needed, and use or save the translated content for their purposes.

To accomplish this, we have to complete all the activities listed below,

- Initialize Gemini Pro LLM:
 - Generate Gemini Pro API
 - Initialize the pre-trained model
- Interfacing with Pre-trained Model
 - Travel itinerary Generation

- Model Deployment
 - Deploy the application using Streamlit

Prior Knowledge

You must have prior knowledge of the following topics to complete this project.

❖ LLM & Gemini Pro:

A large language model is a type of artificial intelligence algorithm that applies neural network techniques with lots of parameters to process and understand human languages or text using self-supervised learning techniques. Tasks like text generation, machine translation, summary writing, image generation from texts, machine coding, chat-bots, or Conversational AI are applications of the Large Language Model. Examples of such LLM models are Chat GPT by open AI, BERT (Bidirectional Encoder Representations from Transformers) by Google, etc.

<https://www.geeksforgeeks.org/large-language-model-llm/>

<https://cloud.google.com/vertex-ai/docs/generative-ai/learn-resources>

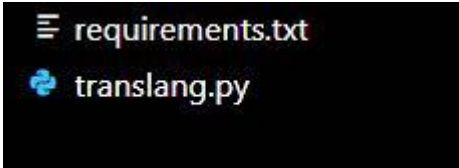
❖ Streamlit:

Basic knowledge of building interactive web applications using Streamlit.

Understanding of Streamlit's UI components and how to integrate them with backend logic.<https://www.datacamp.com/tutorial/streamlit>

Project Structure:

Create the Project folder which contains application file as shown below

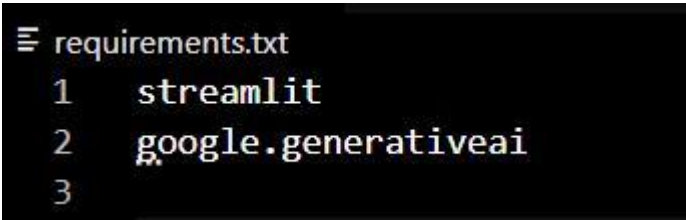


```
requirements.txt  
translang.py
```

MILESTONE 1 : REQUIREMENTS SPECIFICATION

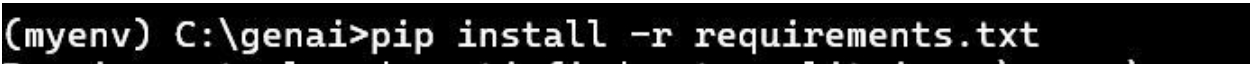
Specifying the required libraries in the requirements.txt file ensures seamless setup and reproducibility of the project environment, making it easier for others to replicate the development environment.

Activity 1 : Create a requirements.txt file to list the required libraries



```
requirements.txt  
1 streamlit  
2 google.generativeai  
3
```

Activity 2 : Install the required libraries.



```
(myenv) C:\genai>pip install -r requirements.txt
```

MILESTONE 2 : INITIALIZING THE MODELS

For initializing the model we need to generate PALM API.

Activity 1: Generate PALM API

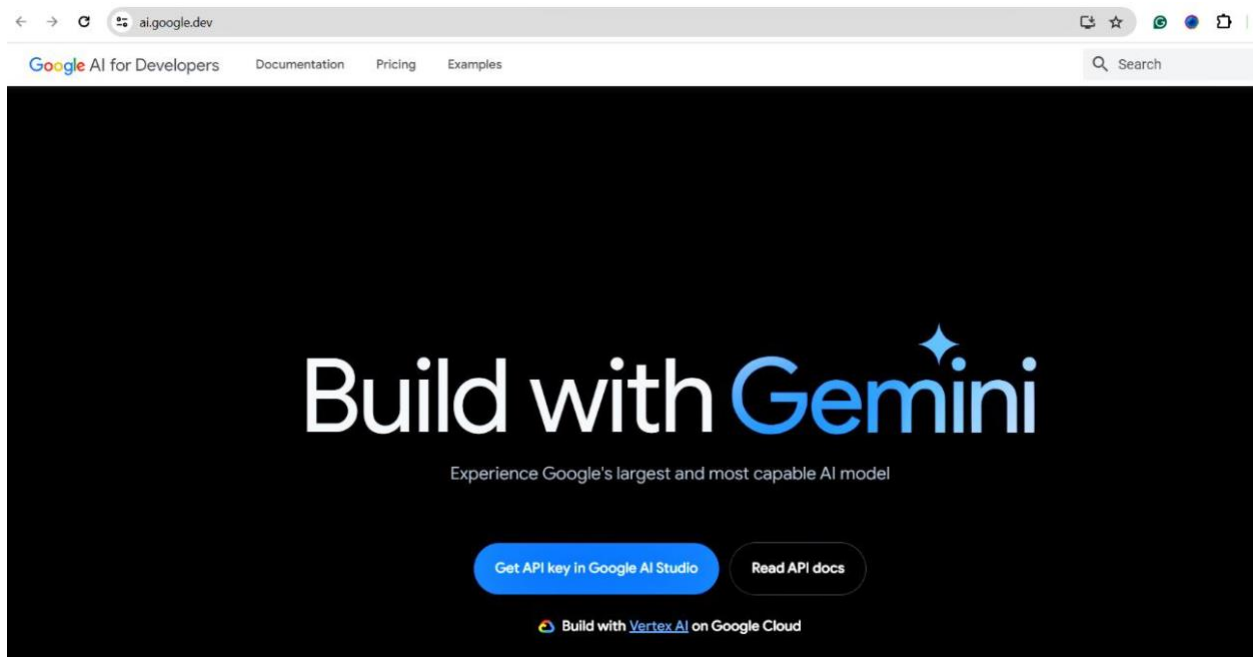
Click on the link (<https://developers.generativeai.google/>).

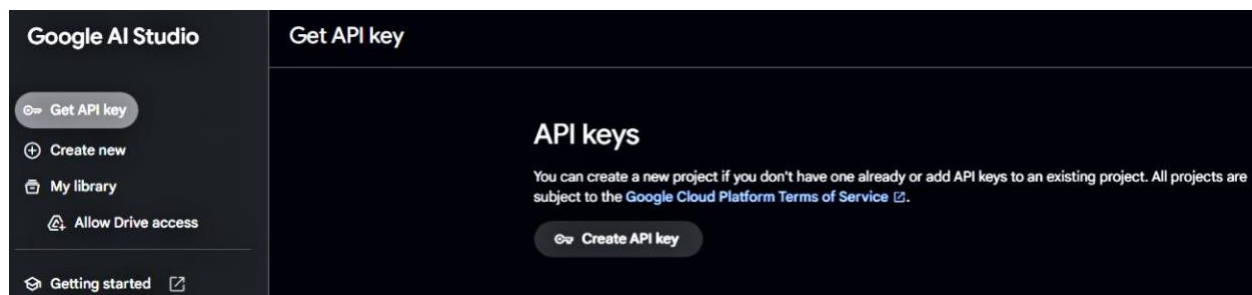
Then click on “Get API key in Google AI Studio”.

Click on “Get API key” from the right navigation menu.

Now click on “Create API key”. (Refer the below images)

Copy the API key.





Activity 2: Initialize the pre-trained model

Activity 2.1 : Import necessary files

```
from dotenv import load_dotenv # typ
import streamlit as st
import os
import google.generativeai as genai
```

- Streamlit, a popular Python library, is imported as st, enabling the creation of user interfaces directly within the Python script.
- Google Generative AI (genai): Imported to interact with the Gemini Pro model.
- The 'load_dotenv' function loads key-value pairs from a '.env' file into environment variables.
- The 'os' module is then used to access and manage these variables.

Activity 2.2: Configuration of the Gemini Pro API

- ❖ Configuring the API key: The configure function is used to set up or configure the Google API with an API key. The provided API key, in this case, is “ AIzaSyB5U5-f1edVl99djSKEcqDoFLcXXXXXX”.

Activity 2.3: Define the model to be used

```
model = genai.GenerativeModel('gemini-1.5-flash')
```

Created an instance of GenerativeModel with the model_name set to “gemini-1.5-flash”.

MILESTONE 3 : INTERFACING WITH PRE-TRAINED MODEL

In this milestone, we will build a prompt template to generate feedback based on the project details entered by the user.

Activity 1: Create a function to generate travel guide

```
# Function to translate text
def translate_text(text, source_language, target_language):
    model = genai.GenerativeModel('gemini-1.5-flash') # Replace with the correct model name
    prompt = (
        f"Translate the following text from {source_language} to {target_language}: {text}"
    )
    response = model.generate_content([prompt])
    return response.text
```

- ❖ `Def translate_text(text, source_language, target_language)::` Defines a function `translate_text` that takes three parameters: `text` (the text to be translated), `source_language` (the language of the input text), and `target_language` (the language to which the text should be translated).

- ❖ `Model = genai.GenerativeModel('gemini-1.5-flash')`: Initializes the generative model with the specified model name ('gemini-1.5-flash'). Replace with the appropriate model name if different.

- ❖ `Prompt = (f"Translate the following text from {source_language} to {target_language}: {text}")`: Constructs a prompt string that instructs the model to translate the provided text from the source language to the target language.

- ❖ `Response = model.generate_content([prompt])`: Sends the constructed prompt to the model to generate the translated content. The `generate_content` method processes the prompt and returns the model's response.

- ❖ `Return response.text`: Returns the translated text from the model's response.

MILESTONE 4 : MODEL DEPLOYMENT

In this milestone, we are deploying the created model using streamlit. Model deployment using Streamlit involves creating a user-friendly web interface, enabling users to interact with the model through a browser. Streamlit provides easy-to-use tools for developing and deploying data-driven applications, allowing for seamless integration of models into web-based applications.

Activity 1 : Give the project title

```
# Initialize Streamlit app
st.set_page_config(page_title="AI-Powered Language Translator", page_icon="🌐")
st.header("🌐 AI-Powered Language Translator")
```

- The main function to start the Streamlit application, allowing users to interact with the web app.
- `st.set_page_config(page_title="AI-Powered Language Translator", page_icon="🌐")`: Sets the app's page title and icon for the browser tab.
- `st.header("🌐 AI-Powered Language Translator")`: Displays a prominent header with the app's title, clarifying its purpose.

Activity 2 : Create fields for user to input data for generating blog

```
# User input for text, source language, and target language
text = st.text_area("📝 Enter text to translate:")
source_language = st.selectbox("🌐 Select source language:", ["English", "Spanish", "French", "German", "Chinese"])
target_language = st.selectbox("🌐 Select target language:", ["English", "Spanish", "French", "German", "Chinese"])

# Translate text button
if st.button("🔄 Translate"):
    if text and source_language and target_language:
        try:
            translated_text = translate_text(text, source_language, target_language)
            st.subheader("📄 Translated Text:")
            st.write(translated_text)
        except Exception as e:
            st.error(f"⚠️ Error: {str(e)}")
    else:
        st.warning("⚠️ Please fill in all fields.")
```

- `text = st.text_area("???? Enter text to translate:")`: Creates a text input field labeled "???? Enter text to translate:" where users can type the text they want to translate. The default value is an empty string.
- `source_language = st.selectbox("???? Select source language:", ["English", "Spanish", "French", "German", "Chinese"])`: Provides a dropdown menu labeled "???? Select source language:" for users to choose the language of the input text.
- `target_language = st.selectbox("???? Select target language:", ["English", "Spanish", "French", "German", "Chinese"])`: Provides a dropdown menu labeled "???? Select target language:" for users to select the language into which the text should be translated.
- `if st.button("???? Translate")`:: Adds a button labeled "???? Translate." When clicked, the app attempts to translate the input text based on the selected source and target languages.
- `if text and source_language and target_language`:: Checks if all required fields (text, source language, and target language) are filled out before proceeding with translation.
- `translated_text = translate_text(text, source_language, target_language)`: Calls the `translate_text` function to get the translated text using the provided inputs.
- `st.subheader("????? Translated Text:")`: Displays a subheader labeled "????? Translated Text:" before showing the translated text.
- `st.write(translated_text)`: Displays the translated text on the web page.
- `else`:: Executes if any input field is empty or missing.
- `st.warning("?? Please fill in all fields.")`: Shows a warning message if any required field is empty, prompting users to complete all fields.

```
if __name__ == "__main__":
    main()
```

- Finally, the `main()` function is called to execute the Streamlit app

Activity 3 : Running the web application

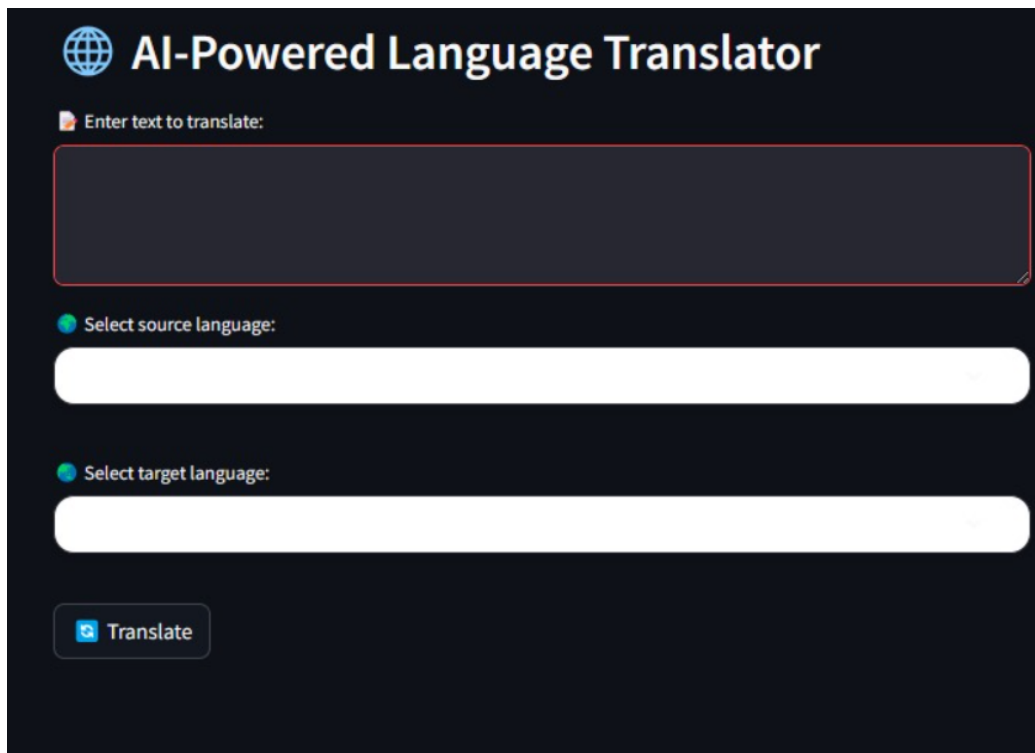
- Open the anaconda prompt from the start menu
- Navigate to the folder where your Python script is.
- Now type “streamlit run app.py” command
- Navigate to the localhost where you can view your web page

```
(s1) D:\smart bridge\Gen AI\translator gen ai>streamlit run translator.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.68.52:8501
```

NOW, THE APPLICATION WILL OPEN IN THE WEB BROWSER



The screenshot shows a web application titled "AI-Powered Language Translator" with a globe icon. It features a text input field labeled "Enter text to translate:", a dropdown menu for "Select source language:", another dropdown menu for "Select target language:", and a "Translate" button with a speech-to-text icon.

AFTER GIVING THE INPUT:

 **AI-Powered Language Translator**

 Enter text to translate:

hello

 Select target language:

French

English

Spanish

French

German

Chinese

 Select source language:

English

English

Spanish

French

German

Chinese

THE OUTPUT GENERATED:

 **AI-Powered Language Translator**

 Enter text to translate:

hello

 Select source language:

 Select target language:

 Translate

 **Translated Text:**

Bonjour

CONCLUSION

The Language Translator project is a user-friendly web application that leverages advanced AI technology to facilitate seamless language translation. By integrating Streamlit with a robust translation API, users can effortlessly translate text between different languages by specifying the source and target languages. The application provides real-time input validation and instant translation results, ensuring an efficient and interactive experience. This project demonstrates the practical application of AI in bridging language barriers, offering an accessible and effective tool for accurate and personalized translation needs.

PHOTOS AND VIDEOS



VIDEO DEMO LINK :

https://drive.google.com/file/d/1Ddx9sRarANFuzMTV9UEtuCC_Yk3mUsIE/view?usp=drivesdk

COMMENTS

FIRST AND FOREMOST, I SINCERELY GRATITUDE TO OUR ESTEEMED INSTITUTE SRI VASAVI DEGREE COLLEGE, FOR GIVING ME THIS OPPORTUNITY TO FULFILL OUR WARM DREAM TO BECOME A GRADUATE. OUR SINCERE GRADITUDE TO OUR LONG-TERM INTERNSHIP GUIDE **SRI L LAKSHMI NARAYANA**, LECTURER DEPARTMENT OF COMPUTER SCIENCE FOR TIMELY COOPERATION AND VALUABLE SUGGESTIONS WHILE CARRYING OUT THIS INTERNSHIP.

I EXPRESS MY SINCERE THANKS AND HEARTFUL GRATITUDE TO **SRI L LAKSHMI NARAYANA**, HOD IN COMPUTER SCIENCE FOR PERMITTING ME TO DO MY PROJECT INTERNSHIP. WE ALSO EXPRESS OUR SPECIAL THANKS TO OUR MENTOR AND COMPUTER LECTURER **Ms. K. RATNA KUMARI** MADAM FOR HER VALUABLE GUIDANCE THROUGHOUT THE INTERNSHIP. I EXPRESS MY SINCERE THANKS AND HEARTFUL GRATITUDE TO **SRI M RAMA KRISHNA**, PRINCIPAL FOR PROVIDING A FAVOURABLE ENVIRONMENT AND SUPPORTING ME DURING THE DEVELOPMENT OF THIS INTERNSHIP.

THANK YOU, SMART BRIDGE

---- TUMMALAPUDI MANIKANTA

TEAM LEADER

SIGNATURE OF THE HOD

SIGNATURE OF THE PRINCIPAL